

강화학습

국립한밭대학교

장수영

강의계획

주	강의내용	날짜
1	OT	09/02
2	강화학습 개념과 기본 원리	09/09
3	강화학습 연구 응용	09/16
4	강화학습 알고리즘 (1)	09/23
5	강화학습 알고리즘 (2)	09/30
6	강화학습 프레임워크 (1)	10/14
7	강화학습 프레임워크 (2)	10/21
8	강화학습 프레임워크 (3)	10/28
9	강화학습 프로젝트 #1	11/04
10	강화학습 프로젝트 #2	11/11
11	강화학습 기말 프로젝트 (1)	11/18
12	강화학습 기말 프로젝트 (2)	11/25
13	강화학습 기말 프로젝트 (3)	12/02
14	강화학습 기말 프로젝트 (4)	12/09
15	강화학습 기말 프로젝트 (5)	12/16

- [공지]
- 10/07(화) 추석연휴
→ 12/16(화) 순연 처리

강의 계획

- 출석 15점
 - 결석 -1점
 - 지각 -0.5점 (강의 시작 후 10분 내)
 - 강의 중 퇴실 적발 -1점
- 과제점수 40점
 - 간단한 프레임워크 활용 과제 3개
 - 주어진 문제 해결
 - 랜덤 팀 구성
- 기말고사 45점
 - 프로젝트 발표 평가
 - 11월 18일 자세한 사항 안내
 - 문제 : 프로젝트 결과 보고 판단
 - 팀 구성 : 랜덤 or 자율

강화학습 프레임워크 (1)	10/14	5점
강화학습 프레임워크 (2)	10/21	5점
강화학습 프레임워크 (3)	10/28	5점
강화학습 프로젝트 #1	11/04	
강화학습 프로젝트 #2	11/11	25점
강화학습 기말 프로젝트 (1)	11/18	
강화학습 기말 프로젝트 (2)	11/25	
강화학습 기말 프로젝트 (3)	12/02	
강화학습 기말 프로젝트 (4) <- 발표일	12/09	45점
강화학습 기말 프로젝트 (5)	12/16	

오프라인 강화학습

- 심층학습이 성공적으로 발전하는데 기여한 큰 요인 중 하나는 large and diverse training dataset.
- 강화학습도 심층학습 발전과 함께 많은 성공을 거두고 있지만,
현재 강화학습은 static dataset이 아닌 active data collection에 의존하기에
데이터를 수집하기에 많은 비용이 들 뿐 아니라 위험할 수 있는 real-world application에 적용하기 어려움.
(robotics, education, healthcare, and autonomous driving)
 - 여전히 RL application이 games와 simulations에 머무는 경우가 많음.
 - Real-world application을 위해서 static datasets 기반 학습에 대한 필요성이 커짐
 - imitation learning, behavior cloning 등의 개념이 있긴 했으나, dataset을 생성한 에이전트(behavior policy)의 성능을 넘어서기 어려움.
- Data-driven RL paradigm인 Offline RL이 등장했고 많은 주목을 받는 중.
- Behavior policy (optimal, sub-optimal) 가 환경과 상호작용하며 축적한 static dataset만을 이용해 정책을 학습하는 것
- 2018년말 BCQ(Batch-Constrained Q-learning)가 등장했고, 많은 주목을 받으며, 많은 연구자들이 offline RL에 집중하는 계기를 제공함.

오프라인 강화학습

- Offline RL의 필요성
 - 많은 학습 데이터셋이 존재함 (예. Physical robot 학습 위한 데이터셋)
 - 실환경에서 exploration하기에는 위험함 (예. Physical robot 경우, robot 혹은 주변 물체들이 손상될 수 있음)
 - 강화학습 알고리즘들의 순수 exploitation 성능 비교가 가능함.
- (online) off-policy RL
 - 이론적으로 off-policy는 다른 policy를 사용해 얻은 경험 데이터를 이용해서 학습 가능
 - offline datasets만으로 off-policy RL 학습 시 대개 실패함.
 - off-policy라고 하더라도 erroneous behavior를 보정하기 위해 온라인 상호작용이 필요함.
 - 단순히 데이터셋을 모아서 off-policy RL 알고리즘 돌리는 것으로 offline RL부족함.
- Offline RL의 핵심 문제 중 하나는 static dataset 이용해
 - 1) generalization 능력 습득하는 것
 - 2) Outside of distribution (OOD) 에 위치한 행동을 피하는 것사이 균형을 맞추는 것

오프라인 강화학습

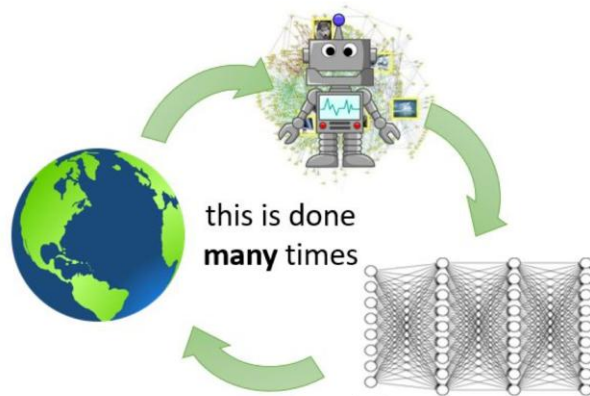
Reinforcement Learning with Online Interactions



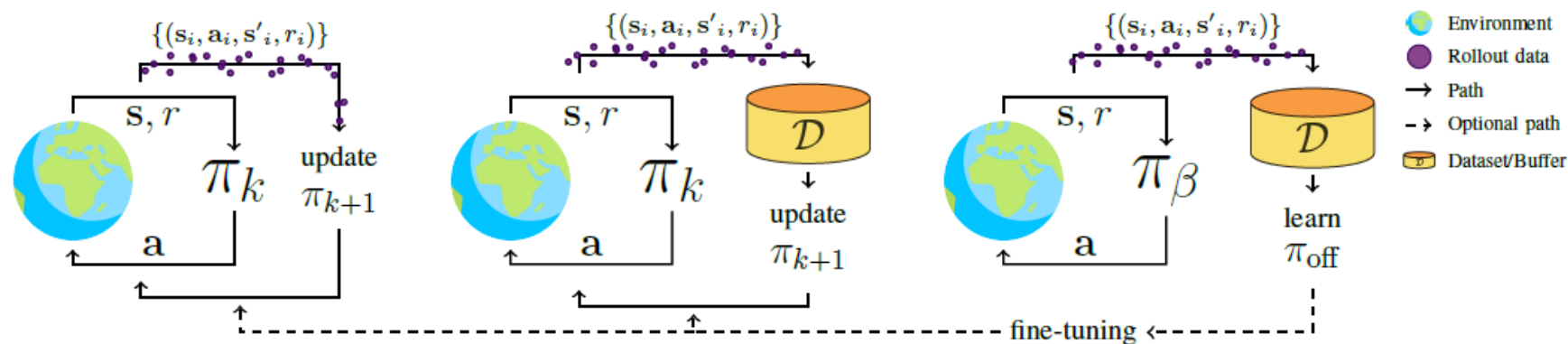
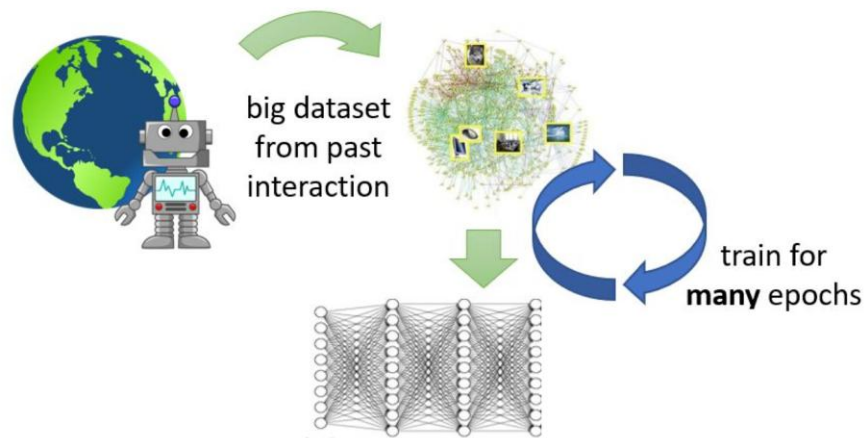
Offline Reinforcement Learning



reinforcement learning



fully off-policy/offline reinforcement learning

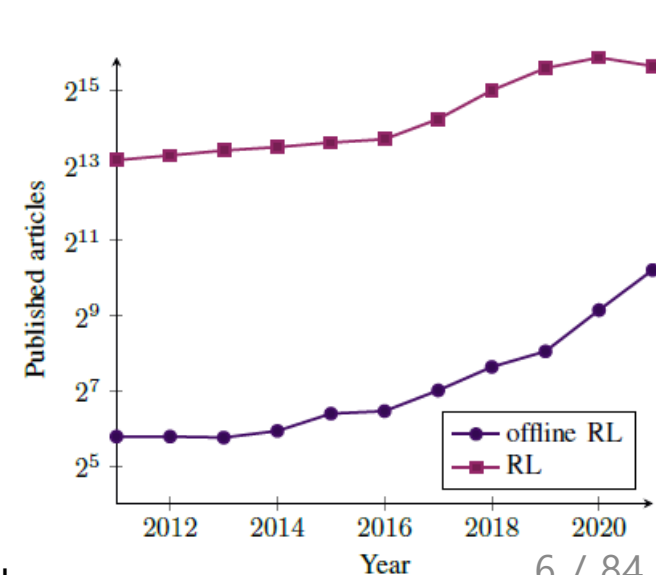


(a) online reinforcement learning

(b) off-policy reinforcement learning

(c) offline reinforcement learning

- Static dataset을 이용해서 정책 학습 후,
이 offline policy 를 환경과 상호작용하며 tuning 할 수 있음.
- 어느 정도 학습이 되었다면, 비용과 위험성 측면에서 많이 개선되었을 것으로 판단



오프라인 강화학습

		강화학습	오프라인 강화학습
환경과 상호작용		Trial-and-error 기반	x
Exploration vs. Exploitation		Exploration-Exploitation Tradeoff 고려	Exploitation만 고려
		오프라인 강화학습	모방학습
공통		Exploration 없이 고정된 데이터셋만으로 학습	
차이	기반 알고리즘	Off-policy 강화학습 (예. Q-Learning)	지도학습
	데이터셋 수준	상대적으로 낮은 수준의 demonstrator로부터의 데이터셋	최적, 혹은 높은 수준의 demonstrator로부터의 데이터셋
	보상 값	보상 값 고려 함 (원 보상 값 사후 처리 고려)	보상 값 고려 안 함
	데이터셋 Labeling	전문가/비전문가 Labeling 필요 없음	전문가/비전문가 Labeling 필요
	Demonstrator 대비 학습 수준	Demonstrator보다 높은 성능 가능	Demonstrator보다 높은 성능 어려움

강화학습 기말 프로젝트

Offline RL with RLlib

프로젝트 개요

- 목표
 - RLlib을 활용해 오프라인 강화학습 기반 에이전트 학습 후 온라인 강화학습 기반 파인튜닝
- 핵심 실습 요소
 - 0. 오프라인 강화학습 예제 따라하기 (~10분 / 구글 클래스룸 코드 제공)
 - 1. MuJoCo HalfCheetah-v5 에 예제 적용하기 (결과물 : 오프라인 강화학습 기반 학습된 에이전트)
 - 2. 오프라인 강화학습 기반 학습된 에이전트에 온라인 강화학습 기반 파인튜닝
 - 3. Minari 데이터셋 중 atari/breakout/expert-v0 을 다운로드 받아서 RLlib 오프라인 강화학습 데이터 포맷에 맞게 변환
 - 4. ALE/Breakout-v5 에 오프라인 강화학습 기반 에이전트 학습 후 온라인 강화학습 기반 파인튜닝 수행
- 환경
 - RLlib (Ray v2.50.1)
 - Gymnasium 1.2.1
 - MuJoCo 3.3.7

프로젝트 개요

- 팀
 - 3명 이내 자율 구성
 - 혼자도 가능하나 되도록 팀 추천

- 제출물(압축본)
 - 수집 데이터
 - 결과보고서
 - 코드
 - 발표자료 (12/08 23:59)

항목	배점	내용
실습 요소 1	5	올바르게 수행되었는 지 여부
실습 요소 2	5	올바르게 수행되었는 지 여부
실습 요소 3	5	올바르게 수행되었는 지 여부
실습 요소 4	10	올바르게 수행되었는 지 여부
보고서 완성도	10	질적 우수성 (얼마나 많은 내용이 알차게 담겼는가) 미적 우수성 (형태, 구성, 표, 그림 등)
추가 점수	10	기본 실습 요소에서 +@ 에 따른 가점 부여. - 다양한 오프라인 강화학습 알고리즘 시도 - 다양한 온라인 강화학습 알고리즘 시도 - 다양한 하이퍼파라미터 튜닝 시도
총점	45	

오프라인 강화학습 예제 따라하기

- <https://docs.ray.io/en/latest/rllib/rllib-offline.html>
- 1. Example: Training an expert policy
 - <https://docs.ray.io/en/latest/rllib/rllib-offline.html#example-training-an-expert-policy>
 - PPO 기반으로 CartPole-v1 학습
 - 기존 프로젝트 코드 사용해도 문제는 없음.
 - 기존과 달리 RLlib Tune 기반으로 학습 수행. 추천 학습 방식.

```
Trial status: 1 TERMINATED
Current time: 2025-11-17 21:46:44. Total running time: 1min 17s
Logical resource usage: 3.0/20 CPUs, 0/1 GPUs (0.0/1.0 accelerator_type:G)
```

Trial name led_lifetime	status	iter	total time (s)	...lls_per_iteration	..._samp
PPO_CartPole-v1_4b60a_00000 176000	TERMINATED	44	69.5634	1	

```
(PPO(env=CartPole-v1; env-runners=2; learners=0; multi-agent=False) pid=49384) Checkpoint successfully creat
ed at: Checkpoint(filesystem=local, path=/home/pz1004/ray_results/docs_rllib_offline_pretrain_ppo/PPO_CartPo
le-v1_4b60a_00000_0_2025-11-17_21-45-27/checkpoint_000043)
```

```
Best checkpoint path: /home/pz1004/ray_results/docs_rllib_offline_pretrain_ppo/PPO_CartPole-v1_4b60a_00000_0
_2025-11-17_21-45-27/checkpoint_000043
```

오프라인 강화학습 예제 따라하기

- 1. Example: Training an expert policy
 - ray_results 폴더 내 Tuner name에 지정된 폴더 내 저장됨.

```
(drl) pz1004@pz1004:~/Workspaces/drl$ ll /home/pz1004/ray_results/docs_rllib_offline_pretrain_ppo/
total 27372
drwxr-xr-x  6 pz1004 pz1004    4096 Nov 17 21:45 ./
drwxr-xr-x 103 pz1004 pz1004   12288 Nov 17 21:16 ../
-rw-r--r--  1 pz1004 pz1004      0 Nov 17 21:45 .validate_storage_marker
drwxr-xr-x 74 pz1004 pz1004    4096 Nov 17 21:18 PPO_CartPole-v1_436df_00000_0_2025-11-17_21-16-36
drwxr-xr-x 46 pz1004 pz1004    4096 Nov 17 21:46 PPO_CartPole-v1_4b60a_00000_0_2025-11-17_21-45-27
drwxr-xr-x 811 pz1004 pz1004   36864 Nov 17 21:40 PPO_CartPole-v1_a7679_00000_0_2025-11-17_21-19-24
drwxr-xr-x 62 pz1004 pz1004    4096 Nov 17 21:44 PPO_CartPole-v1_e67cc_00000_0_2025-11-17_21-42-38
-rw-r--r--  1 pz1004 pz1004   16131 Nov 17 21:18 basic-variant-state-2025-11-17_21-16-36.json
-rw-r--r--  1 pz1004 pz1004   16133 Nov 17 21:41 basic-variant-state-2025-11-17_21-19-24.json
-rw-r--r--  1 pz1004 pz1004   16118 Nov 17 21:44 basic-variant-state-2025-11-17_21-42-38.json
-rw-r--r--  1 pz1004 pz1004   16106 Nov 17 21:46 basic-variant-state-2025-11-17_21-45-27.json
-rw-r--r--  1 pz1004 pz1004  2154617 Nov 17 21:18 experiment_state-2025-11-17_21-16-36.json
-rw-r--r--  1 pz1004 pz1004 22274139 Nov 17 21:41 experiment_state-2025-11-17_21-19-24.json
-rw-r--r--  1 pz1004 pz1004  2051610 Nov 17 21:44 experiment_state-2025-11-17_21-42-38.json
-rw-r--r--  1 pz1004 pz1004 1390091 Nov 17 21:46 experiment_state-2025-11-17_21-45-27.json
-rw-r--r--  1 pz1004 pz1004    9296 Nov 17 21:45 tuner.pkl
```

```
# Define the Tuner.
tuner = tune.Tuner(
    "ppo",
    param_space=config,
    run_config=tune.RunConfig(
        stop={
            metric: 450.0,
        },
        name="docs_rllib_offline_pretrain_ppo",
        verbose=2,
        checkpoint_config=tune.CheckpointConfig(
            checkpoint_frequency=1,
            checkpoint_at_end=True,
        ),
    ),
)

results = tuner.fit()
```

- 모델이 저장된 checkpoint 들과 학습 결과 확인 위한 파일들이 저장된 것 확인 가능함.

```
drwxr-xr-x 4 pz1004 pz1004    4096 Nov 17 21:46 checkpoint_000042/
drwxr-xr-x 4 pz1004 pz1004    4096 Nov 17 21:46 checkpoint_000043/
-rw-r--r-- 1 pz1004 pz1004 404256 Nov 17 21:46 events.out.tfevents.1763383533.pz1004
-rw-r--r-- 1 pz1004 pz1004  12538 Nov 17 21:46 params.json
-rw-r--r-- 1 pz1004 pz1004   9931 Nov 17 21:46 params.pkl
-rw-r--r-- 1 pz1004 pz1004  63542 Nov 17 21:46 progress.csv
-rw-r--r-- 1 pz1004 pz1004 723711 Nov 17 21:46 result.json
```


오프라인 강화학습 예제 따라하기

- 2. Example: Record expert data to local disk
 - <https://docs.ray.io/en/latest/rllib/rllib-offline.html#example-record-expert-data-to-local-disk>
 - PPO 기반으로 학습된 CartPole-v1 에이전트 불러와서 평가 수행하며 오프라인 강화학습 데이터 획득

```
(pid=126305) ✓ Dataset dataset_151_0 execution finished in 0.09 seconds: 100%| 10.0/10.0 [00:00<00:00, 9
{'env_runners': {'agent_episode_return_mean': {'default_agent': 497.72}, 'env_reset_timer': np.float64(0.000
2006639529332767), 'env_step_timer': np.float64(4.2862210325701514e-05), 'env_to_module_connector': {'connec
tor_pipeline_timer': np.float64(8.04478275517602e-05), 'timers': {'connectors': {'add_observations_from_epis
odes_to_batch': np.float64(6.100527060466293e-06), 'add_states_from_episodes_to_batch': np.float64(1.2218535
86222086e-06), 'add_time_dim_to_batch_and_zero_pad': np.float64(2.0616561993933034e-06), 'batch_individual_i
tems': np.float64(1.2381279417052118e-05), 'numpy_to_tensor': np.float64(2.5679311950426356e-05)}}}, 'env_to
_module_sum_episodes_length_in': np.float64(404.2802310597345), 'env_to_module_sum_episodes_length_out': np.
float64(404.2802310597345), 'episode_duration_sec_mean': 0.2653141077526379, 'episode_len_max': 500, 'episod
e_len_mean': 497.72, 'episode_len_min': 386, 'episode_return_max': 500.0, 'episode_return_mean': 497.72, 'ep
isode_return_min': 386.0, 'module_episode_return_mean': {'default_policy': 497.72}, 'module_to_env_connector
': {'connector_pipeline_timer': np.float64(0.0002643885820300627), 'timers': {'connectors': {'get_actions':
np.float64(0.00012781341634593603), 'listify_data_for_vector_env': np.float64(1.8854711507620494e-05), 'norm
alize_and_clip_actions': np.float64(2.034793591555894e-05), 'remove_single_ts_time_rank_from_batch': np.floa
t64(1.3419851375052325e-06), 'tensor_to_numpy': np.float64(3.618848095863433e-05), 'un_batch_to_individual_i
tems': np.float64(1.0871866735959752e-05)}}}, 'num_agent_steps_sampled': {'default_agent': 322842.0}, 'num_a
gent_steps_sampled_lifetime': {'default_agent': 386923.9999999999}, 'num_env_steps_sampled': 322842.0, 'num_
env_steps_sampled_lifetime': 29386.0, 'num_episodes': 650.0, 'num_episodes_lifetime': 830.0000000000002, 'nu
m_module_steps_sampled': {'default_policy': 322842.0}, 'num_module_steps_sampled_lifetime': {'default_policy
': 386923.9999999999}, 'rlmodule_inference_timer': np.float64(5.400252821193489e-05), 'sample': np.float64(6
.208308454864838), 'time_between_sampling': np.float64(0.728226629801236), 'weights_seq_no': 50.0, 'recordin
g_buffer_size': np.float64(4.912855302068928)}}
(pid=126306) ✓ Dataset dataset_157_0 execution finished in 0.07 seconds: 100%| 10.0/10.0 [00:00<00:00, 9
(pid=126305) ✓ Dataset dataset_162_0 execution finished in 0.08 seconds: 100%| 10.0/10.0 [00:00<00:00, 9
(OfflineSingleAgentEnvRunner pid=126305) Registered dataset logger for dataset dataset_162_0 [repeated 2x ac
ross cluster]
(OfflineSingleAgentEnvRunner pid=126305) Starting execution of Dataset dataset_162_0. Full logs are in /tmp/
ray/session_2025-11-17_23-59-30_317212_125533/logs/ray-data [repeated 2x across cluster]
(OfflineSingleAgentEnvRunner pid=126305) Execution plan of Dataset dataset_162_0: InputDataBuffer[Input] ->
TaskPoolMapOperator[Write] [repeated 2x across cluster]
(OfflineSingleAgentEnvRunner pid=126305) ✓ Dataset dataset_162_0 execution finished in 0.08 seconds [repea
ted 3x across cluster]
(OfflineSingleAgentEnvRunner pid=126305) Data sink Parquet finished. 10 rows and 237.8KB data written. [repe
ated 3x across cluster]
```

오프라인 강화학습 예제 따라하기

- 3. Example: Training on previously saved experiences
 - <https://docs.ray.io/en/latest/rllib/rllib-offline.html#example-training-on-previously-saved-experiences>
 - 획득한 오프라인 강화학습 데이터 기반해서 에이전트 학습
 - 여기서는 Behavior Cloning 사용

PPO 기반으로 처음부터 학습하는 경우,
69.56초 걸렸음.

CartPole 같이 엄청 쉬운 환경에서는
처음부터 학습하는 경우가 나올 수도 있으나,
대개 오프라인 강화학습의 학습 시간이 짧음.

```
Trial BC_CartPole-v1_adfa2_000000 finished iteration 399 at 2025-11-18 00:36:26. Total running time: 3min 34s
```

Trial BC_CartPole-v1_adfa2_000000 result	
evaluation/env_runners/episode_return_mean	451.89
num_env_steps_sampled_lifetime	0

```
Trial BC_CartPole-v1_adfa2_000000 saved a checkpoint for iteration 399 at: (local)/home/pz1004/ray_results/docs_rllib_offline_bc/BC_CartPole-v1_adfa2_000000_0_2025-11-18_00-32-51/checkpoint_000000
```

```
Trial BC_CartPole-v1_adfa2_000000 completed after 399 iterations at 2025-11-18 00:36:26. Total running time: 3min 34s
```

```
2025-11-18 00:36:26,212 INFO tune.py:1009 -- Wrote the latest version of all result files and experiment state to '/home/pz1004/ray_results/docs_rllib_offline_bc' in 0.0105s.
```

```
Trial status: 1 TERMINATED
```

```
Current time: 2025-11-18 00:36:26. Total running time: 3min 34s
```

```
Logical resource usage: 2.0/20 CPUs, 0/1 GPUs (0.0/1.0 accelerator_type:G)
```

Trial name	status	iter	total time (s)	...lls_per_iteration	...sampl
BC_CartPole-v1_adfa2_000000	TERMINATED	399	201.216	1	

```
(BC(env=CartPole-v1; env-runners=0; learners=0; multi-agent=False) pid=166169) Checkpoint successfully created at: Checkpoint(filesystem=local, path=/home/pz1004/ray_results/docs_rllib_offline_bc/BC_CartPole-v1_adfa2_000000_0_2025-11-18_00-32-51/checkpoint_000000)
```

```
(MapWorker(MapBatches(OfflinePreLearner)) pid=166733) DeprecationWarning: `RLModule(config=[RLModuleConfig object])` has been deprecated. Use `RLModule(observation_space=.., action_space=.., inference_only=.., model_config=.., catalog_class=..)` instead. This will raise an error in the future! [repeated 3x across cluster]
```

```
(Ray deduplicates logs by default. Set RAY_DEDUP_LOGS=0 to disable log deduplication, or see https://docs.ray.io/en/master/ray-observability/user-guides/configure-logging.html#log-deduplication for more options.)
```

[참고] Offline RL

- Ray.Data has been integrated into RLlib, enabling large-scale data ingestion for offline RL and behavior cloning (BC) workloads.
- See here for a basic tuned example for the behavior cloning algo (https://github.com/ray-project/ray/blob/master/rllib/tuned_examples/bc/cartpole_bc.py) and here for how to pre-train a policy with BC, then finetuning it with online PPO (https://github.com/ray-project/ray/blob/master/rllib/examples/offline_rl/train_w_bc_finetune_w_ppo.py)
- 기타 참고
 - https://github.com/ray-project/ray/tree/master/rllib/examples/offline_rl

[참고] Minari

- A dataset API for Offline Reinforcement Learning
 - <https://github.com/Farama-Foundation/Minari>
 - minari.farama.org

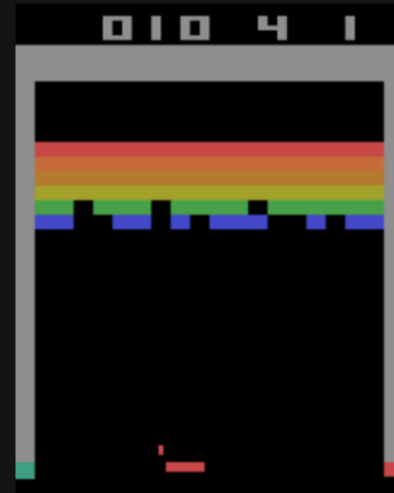
Minari datasets in hf://farama-minari				
Name	# Episod...	# Ste...	Size	Author
mujoco/... Group with 28 datasets	54K	23M	24.26 GB	Kallinteris Andreas
minigrid/... Group with 170 datasets	170K	4M	7.51 GB	Omar G. Younis
atari/... Group with 57 datasets	570	3M	11.98 GB	Omar G. Younis
metaworld/... Group with 50 datasets	2K	1M	474.9 MB	Omar G. Younis
webagents/weblinx-v0	969	24K	135.1 MB	Zdeněk Kasner, Siva Reddy, Xing Han ...

[참고] Minari

- <https://minari.farama.org/datasets/atari/breakout/expert-v0/>
- Expert data generated by CleanRL PPO Impala. The training code uses **standard Atari tricks**, such as frame stacking, grayscale, and resizing. However, **the dataset observations are not preprocessed**. You can find the training code, model, and metrics on CleanRL's HuggingFace repository.
- https://github.com/Farama-Foundation/minari-dataset-generation-scripts/blob/main/scripts/atari/create_datasets.py

```
def make_env(env_id):  
    env = gym.make(env_id, repeat_action_probability=0, max_episode_steps=108000 // 4)  
    data_collector = DataCollector(env)  
    env = gym.wrappers.ResizeObservation(data_collector, (84, 84))  
    env = gym.wrappers.GrayscaleObservation(env)  
    env = gym.wrappers.FrameStackObservation(env, 4)  
    return env, data_collector
```

Expert



Atari tricks 적용 전에
DataCollector 씌움.

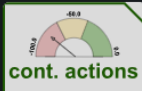
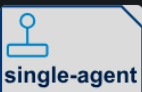
[참고] Minari

- <https://minari.farama.org/datasets/atari/breakout/expert-v0/>

Dataset Specs		<pre>import minari dataset = minari.load_dataset('atari/breakout/expert-v0') env = dataset.recover_environment() obs, info = env.reset() print(obs.shape)</pre>
Total Steps	23075	
Total Episodes	10	<pre>(drl) pz1004@pz1004:~/Workspaces/drl\$ python minari_test.py A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138) [Powered by Stella] (210, 160, 3)</pre>
Dataset Observation Space	Box(0, 255, (210, 160, 3), uint8)	
Dataset Action Space	Discrete(4)	
Algorithm	CleanBA PPO Impala	
Author	Omar G. Younis	
Email	omar@farama.org	
Code Permalink	https://github.com/Farama-Foundation/minari-dataset-generation-scripts	
Minari Version	0.5.2 (supported)	
Download	minari download atari/breakout/expert-v0	

RLlib Offline RL

- <https://docs.ray.io/en/latest/rllib/rllib-algorithms.html>
- https://docs.ray.io/en/latest/rllib/package_ref/algorithm-config.html#rllib-config-offline-data

Offline RL and Imitation Learning			
BC (Behavior Cloning)	 single-agent	 multi-GPU  multi-node multi-GPU	 cont. actions  discrete actions
CQL (Conservative Q-Learning)	 single-agent	 multi-GPU  multi-node multi-GPU	 cont. actions
IQL (Implicit Q-Learning)	 single-agent	 multi-GPU  multi-node multi-GPU	 cont. actions
MARWIL (Monotonic Advantage Re-Weighted Imitation Learning)	 single-agent	 multi-GPU  multi-node multi-GPU	 cont. actions  discrete actions

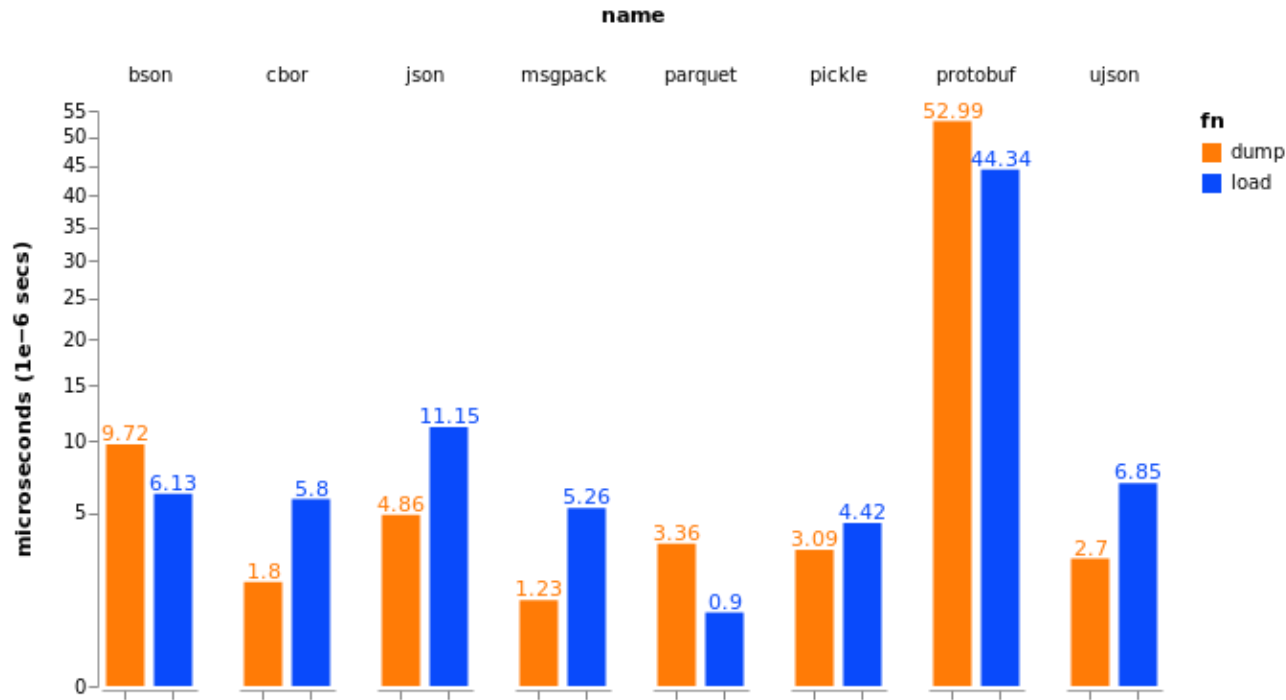
Configuring offline RL specific settings

```
AlgorithmConfig.offline_data(*, input_: str |  
~typing.Callable[~ray.rllib.offline.io_context.IOContext],  
~ray.rllib.offline.input_reader.InputReader] | None =  
<ray.rllib.utils.from_config._NotProvided object>, offline_data_class:  
~typing.Type | None = <ray.rllib.utils.from_config._NotProvided object>,  
input_read_method: str | ~typing.Callable | None =  
<ray.rllib.utils.from_config._NotProvided object>,  
input_read_method_kwargs: ~typing.Dict | None =  
<ray.rllib.utils.from_config._NotProvided object>, input_read_schema:  
~typing.Dict[str, str] | None = <ray.rllib.utils.from_config._NotProvided  
object>, input_read_episodes: bool | None =  
<ray.rllib.utils.from_config._NotProvided object>,  
input_read_sample_batch_size: bool | None =
```

[참고] Python serialization

- Python Serialization Benchmarks (2018.12.31)

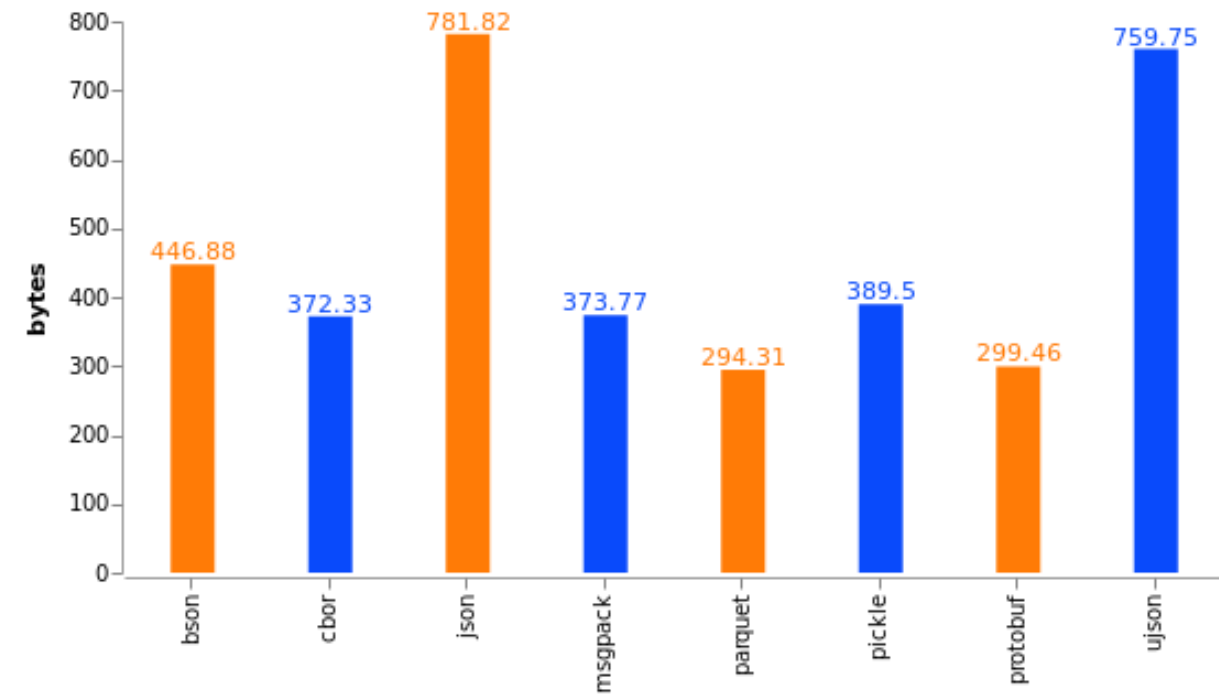
Average Object Serialization Time (1M objects)



msgpack이 serialization은 제일 빠르네
대신 deserialization이 느리네

parquet은 반대네

Average Object Serialized Size (1M objects)



parquet이 용량이 제일 작네